Ders 13 Çalışma Özeti

Ders 13 Çalışma Özeti

Genel Konular

- Dersi dersin hocası Dr. Ahmet Elbir online anlatıyor (12 Mayıs 2025). Konular: `void` işaretçi, dosya işlemlerine giriş (stream, metin ve ikili dosyalar, `fopen` erişim kipleri, dosya konum göstergesi), dosya kopyalama örneği ve dönem projesinin tanıtımı.
- Duyurular: sınav çözümleri ve değerlendirmesi çarşamba günü online yapılacak. Cuma sabahı 9-12 arası ek ders var (uygulamaya ve projeye hazırlık).
- `void` işaretçi (genel işaretçi)
  - Her türden değişkenin adresini tutabilen genel (generic) işaretçidir: `void *ptr;`. `int*`, `float*`, `char*` veya bir yapı işaretçisi yerine kullanılabilir.
  - Ne işe yarar? Türden bağımsız (generic) fonksiyonlar yazmayı sağlar. Örneğin `printIntArray`, `printFloatArray` gibi ayrı ayrı fonksiyonlar yerine tek bir `printArray`. İleride thread konusunda da kullanılacak.
  - `malloc` ve `calloc` geriye `void*` döndürür. Bu sayede her tür için ayrı bir `malloc` gerekmez. Sonuç istenen türe dönüştürülür (cast): `(int*)malloc(n * sizeof(int))`.
  - `void*` doğrudan dereferans edilemez. `*ptr = 10;` derleme hatası verir. Önce asıl türüne çevrilir: `*(int*)ptr = 10;`.
  - Örnek: `ptr = &i; *(int*)ptr = 10;` sonrası `i` 10 olur. `ptr = &f;` sonrası `*(float*)ptr` ile `f` okunur.
  - Dizi üzerinde: `ptr = a;` ile dizinin başı tutulur. `ptr++` adresi yalnızca 1 bayt artırdı ve yanlış bir değer okundu. `int` dizisinde bir sonraki eleman 4 bayt ileridedir. Hoca `ptr += sizeof(int)` yazarak ikinci elemana ulaştı.
  - `void` (dönüş değeri yok) ile `void*` (genel işaretçi) tamamen farklı şeylerdir.
- Neden dosya?
  - Şimdiye kadar girdi klavyeden, çıktı ekrana yapıldı. Program kapanınca veriler kaybolur, çünkü program uçucu bir bellek olan RAM'de çalışır.
  - Kalıcılık için veriler diske (HDD/SSD) yazılır. Bilgisayar kapansa da veriler kalır.
  - Disk işletim sisteminin denetimindedir. Dosyalara erişim, yetkisiz bellek erişimindeki segmentation fault gibi işletim sistemi kurallarına tabidir. İşletim sistemi bir klasöre veya dosyaya erişimi engelleyebilir.
  - C kütüphanesi, işletim sisteminin girdi/çıktı servislerini kolay kullanmayı sağlayan fonksiyonlar sunar (dosya açma, okuma, yazma, silme, kopyalama).
  - C kütüphanesi tamponlu (buffered) girdi/çıktı kullanır. `scanf("%d")` sonrası `%c` okunamaması sorununun nedeni de tamponda kalan karakterdir.
- Stream (akış)
  - Program (RAM) ile dosya (disk) veya cihaz arasında kurulan mantıksal bağlantıdır. Sıralı bir bayt dizisidir.
  - Cihaz da olabilir: ağa veri göndermek bir yazma, mikrofondan veya tarayıcıdan veri almak bir okumadır. İşletim sisteminde en altta genel `read` ve `write` işlemleri vardır.
  - Her programa otomatik açılan üç stream: standart girdi (`stdin`, klavye), standart çıktı (`stdout`, ekran) ve standart hata (`stderr`).
  - `printf` = `fprintf(stdout, ...)`, `scanf` = `fscanf(stdin, ...)`. `fprintf` ve `fscanf` farklı hedeflere yazıp okuyabilen genel hâllerdir.
  - Dosya, `stdio.h` içindeki `FILE` yapısı üzerinden bir işaretçiyle (`FILE *fp`) yönetilir.
- Metin (text) ve ikili (binary) dosyalar
  - Metin dosyası: satırlardan oluşur, satırlar `\n` ile biter. Herhangi bir metin editöründe insanın okuyabileceği şekilde görünür (ör. `.c`, `.txt`, `.csv`).

  - CSV (comma separated values): sütunlar virgülle ayrılır, her satır bir kayıttır (record). TSV'de ayraç sekmedir. Kendi ayracınızı da belirleyebilirsiniz (ör. **).
  - Metin dosyasının dezavantajları: her karakter bir bayt yer tutar, büyük verilerde dosya çok büyür. Veride ayraç karakteri (ör. virgül) geçerse karışıklık olur.
  - İkili dosya: veriler kodlanmış olarak saklanır, editörde anlamsız görünür (ör. `.exe`, `.png`, `.jpg`). Aynı ekran görüntüsünün PNG ve JPEG hâlleri farklı biçimlerde kodlanır. İkili biçim, bilginin doğrudan okunmasını ve kodun çalınmasını zorlaştırır.
- Dosya konum göstergesi (file position indicator)
  - Dosya işaretçisi (`FILE*`) dosyayı gösterir. İçindeki dosya konum göstergesi ise dosyada şu an nerede olunduğunu gösterir, komut satırındaki imleç gibi.
  - Dosya açılınca başa konumlanır. Her okuma veya yazmada okunan/yazılan bayt kadar ileri gider. Geri gitmek için özel fonksiyonlar vardır (`fseek`, `rewind`).
- `fopen` ve erişim kipleri
  - `FILE *fp = fopen("deneme.txt", "r");`. İlk parametre dosya adı, ikincisi erişim kipi. Başarısızsa `NULL` döner.
  - `"r"`: var olan metin dosyasını okumak için açar, baştan okur.
  - `"w"`: yazmak için yeni dosya oluşturur. Dosya varsa içi tamamen silinir (truncate). Dikkatli kullanılmalı.
  - `"a"`: ekleme (append) kipi. Mevcut veri korunur, yazma her zaman dosyanın sonuna yapılır.
  - `"r+"`: var olan dosyayı okuma ve yazma için açar, konum başta.
  - `"w+"`: okuma ve yazma için yeni dosya oluşturur, varsa içini siler.
  - `"a+"`: dosyayı açar veya oluşturur, her yerden okunabilir ama yalnızca sona yazılır.
  - İkili kip için sona `b` eklenir: `"rb"`, `"wb"`, `"ab"`, `"r+b"`, `"w+b"`.
  - Açmadan önce dosyanın var olması gereken kipler `r` ve `r+`'dır. `w` ve `w+` eski veriyi siler.
- Okuma ve yazma birimleri
  - Karakter (bayt): `getc`/`putc` (`fgetc`/`fputc`).
  - Satır (string): `fgets`/`fputs`.
  - Blok: `fread`/`fwrite`.
  - Biçimli: `fprintf`/`fscanf`.
  - Satır okumak için karakter karakter okumak yavaştır. Disk, RAM'den çok daha yavaştır. Her karakter için diske gidip gelmek büyük gecikmedir.
  - "Bir dosya nasıl okunur? Nasıl yazıldıysa öyle." Örneğin CSV dosyası satır satır okunur, sonra virgüllere göre ayrıştırılır (parse). "Bir dosya nasıl yazılır? Nasıl istiyorsanız, ama mühendislik yaparak." Veride virgül olabilecekse ayraç olarak noktalı virgül seçmek daha iyi olabilir.
  - Dosya biçimleri genellikle başlık (header), veri (data) ve bazen son ek (footer) bölümlerinden oluşur. Örnekler: PGM (gri seviye görüntü, başlıkta tür, genişlik, yükseklik, en büyük değer), PPM (renkli), BMP (başlıkta imza, dosya boyutu, genişlik, yükseklik, bit sayısı…).
  - Dosya biçimini tasarlamak geleceği düşünmeyi gerektirir. IPv4 (32 bit adres) yetmiyor, IPv6 (128 bit) daha iyi, ama bütün cihazlar ve ağ altyapısı IPv4 biçimine göre kurulduğu için geçiş zordur.
  - `fopen`'dan sonra mutlaka `fclose`. Dosya kapatılmazsa son yazılan veriler dosyaya yansımayabilir. "`malloc` yapınca hemen `free`'yi yazdığımız gibi, `fopen` yapınca hemen `fclose`'u yazıyoruz."
- Derste yazılan kodlar
  - Dosya açma kontrolü: `if ((fp = fopen("deneme.txt", "r")) == NULL) { fprintf(stderr, "Dosya açma hatası"); } else { ...; fclose(fp); }`. Atama operatörü parantez içinde olduğu için önce atama, sonra karşılaştırma yapılır.
  - `printf` ile `stdout`'a, `fprintf(stderr, ...)` ile hata akışına yazıldı. Hata akışı bazı dillerde (ör. Java) ayrı renkte veya farklı sırada görünür.
  - Yol yazarken ters bölü iki kez yazılır: `"C:\\deneme.txt"`.
  - `FILE* openFile(char *fName, char *mode)`: dosyayı açıp işaretçisini döndüren yardımcı fonksiyon. `"w"` kipiyle açılan dosyanın eski içeriğinin silindiği gösterildi.
  - Dosya kopyalama: `while ((c = getc(fp)) != EOF) putc(c, fp2);`. İlk denemede `feof` ile kurulan döngü sonsuza girdi ve dosya açılamadı. Hoca bunun arka planda dosyayı tutan başka bir programdan kaynaklandığını düşündü, programları kapatıp yeniden çalıştırınca kopyalama çalıştı. `c` değişkeninin `int` olması önemlidir.
- Dönem projesi (Online Yıldız'da ilan edildi)

- Kütüphane otomasyonuna benzer bir veri tabanı uygulaması. Bilgiler hem CSV dosyalarında kalıcı olarak, hem de programda yapılar (struct) ve bağlı listelerde tutulacak.
- Beş CSV dosyası: yazarlar (ad, soyad, otomatik verilen yazar ID), öğrenciler (ad, soyad, öğrenci numarası, kütüphane puanı), kitaplar (ad, ISBN, adet; her nüsha "ISBN_1" gibi etiketlenir), kitap-yazar eşleştirmesi (ISBN ile yazar ID; yazar silinirse -1) ve ödünç alma işlemleri (nüsha, öğrenci, tarih, işlem türü 0 ödünç / 1 teslim).
- Kitap 15 günden geç teslim edilirse öğrencinin puanı düşer, cezalı öğrenci listesi çıkarılır.
- İşlemler: öğrenci ekleme/silme/güncelleme/görüntüleme, kitap ve yazar işlemleri, yazar-kitap eşleştirme. İşlemlerin çoğu ID üzerinden. "OBS'nin küçük ölçekli hâli gibi."
- Teslim: kod, rapor ve CSV dosyaları Online Yıldız'a tek bir RAR/ZIP içinde. Bir de video çekilecek, YouTube linki rapora eklenecek. Google Drive, OneDrive, GitHub linki kabul edilmiyor. Video yoksa çok düşük puan alınır.

Hocanın Özellikle Vurguladığı Kısımlar

- `void*` dereferans edilemez, önce asıl türe çevrilmelidir.
- `"w"` kipi var olan dosyanın içini siler.
- `fopen` sonucu `NULL` mı diye mutlaka kontrol edilmeli. `fopen` yapınca hemen `fclose`'u yazın.
- Bir dosya nasıl yazıldıysa öyle okunur. Dosya biçimini mühendislik yaparak tasarlayın.
- Satır okumak için karakter karakter okumayın, disk yavaştır.
- Projede video zorunlu, link olarak yalnızca YouTube kabul ediliyor.

Kısa Tekrar Notları

- `void *p; *(int*)p = 10;`
- `FILE *fp = fopen("ad", "kip"); if (fp == NULL) ...; fclose(fp);`
- Kipler: `r`, `w` (siler), `a` (sona ekler), `r+`, `w+`, `a+`; ikili için `b`.
- Karakter: `getc`/`putc`. Satır: `fgets`/`fputs`. Blok: `fread`/`fwrite`. Biçimli: `fprintf`/`fscanf`.
- Kopyalama: `while ((c = getc(fp)) != EOF) putc(c, fp2);`
- Üç standart akış: `stdin`, `stdout`, `stderr`.

Detaylı Açıklamalar

Neden `while (!feof(fp))` önerilmez?

- `feof`, yalnızca bir okuma dosya sonunu geçmeye çalıştıktan sonra doğru (sıfırdan farklı) döner. Döngü `while (!feof(fp)) putc(getc(fp), fp2);` biçiminde yazılırsa, son karakterden sonra bir kez daha `getc` çağrılır. Bu çağrı `EOF` (-1) döndürür ve `putc` bunu `0xFF` baytı olarak hedef dosyaya yazar.
- Test (derste yazılmadı): 12 baytlık `"Merhaba YTU\n"` dosyası `getc != EOF` döngüsüyle 12 bayt, `!feof` döngüsüyle 13 bayt (sonda fazladan `ff`) kopyalandı.
- Doğru kalıp derste sonunda kullanılan kalıptır: okuma işleminin sonucunu doğrudan `EOF` ile karşılaştırmak. `c`'nin `char` değil `int` olması gerekir, çünkü `EOF` bir `char` değeri değildir.

Ders sonrası notlar

- **Kodların kontrolü:** `void` işaretçi örneği, dosya açma kontrolü ve kopyalama kodu gcc ile denendi.
- **`feof`'un dönüş değeri:** Derste "dosyanın sonuna geldiysen sıfır değerini elde edersin" dendi. Tersidir: `feof` dosya sonuna ulaşıldıysa sıfırdan farklı, ulaşılmadıysa sıfır döndürür. Bu yüzden döngü `while (!feof(fp))` şeklinde kurulur (ama yukarıda anlatılan sorun nedeniyle önerilmez).
- **`void*` ile adres aritmetiği:** Standart C'de `void*` üzerinde aritmetik (`ptr++`, `ptr += 4`) tanımlı değildir. GCC bunu bir uzantı olarak `char*` gibi (1 bayt) yapar, derste görülen davranış budur. Taşınabilir yazım: `ptr = (char*)ptr + sizeof(int);` veya `ptr = (int*)ptr + 1;` (derste yazılmadı, test edildi). `-Wpedantic` ile derlenince GCC `void*` aritmetiği için uyarı verir.

- **`"a"` kipi:** Slaytta “var olan bir metin dosyasını açar” yazıyor. Dosya yoksa `"a"` kipi dosyayı oluşturur (test edildi). Yazma her zaman sona yapılır.
- **Standart akış numaraları:** Derste girdi 0, çıktı 1, hata 2 denildi. Bunlar işletim sistemi düzeyindeki dosya tanımlayıcılarıdır (file descriptor). C’deki `stdin`, `stdout`, `stderr` ise `FILE*` türünde akışlardır.
- **“Bit okumak”:** Derste ikili dosyalardan “bit okumak”tan söz edildi. Dosya işlemlerinde en küçük okuma birimi bayttır, ikili dosyalarda da öyledir. Tek tek bitlere ulaşmak için bayt okunup bit işlemleri (`&`, `>>`) kullanılır.
- **RAM ve disk hızı:** Derste verilen frekans rakamları (RAM için “10 MHz”, disk için “kHz”) yalnızca büyüklük farkını anlatmak için kullanılan kaba benzetmelerdir. Asıl mesaj doğrudur: diske erişim RAM’e erişimden kat kat yavaştır. SSD’ler HDD’lerden çok hızlıdır ama yine de RAM’den yavaştır.
- **IPv4 adres sayısı:** $2^{32} \approx 4.3$ milyar adres eder. Derste kaba hesap olarak $2^{30} \approx 10^9$ kullanıldı.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.